

MaintPro Industrial OS — Guía Técnica Completa para Entrevista

Autor: Piero Salcedo
Stack: Java 21 · Spring Boot 3.3 · PostgreSQL · Angular 22 · JWT · Spring Security
Repositorios: [Backend](#) · [Frontend](#)

Tabla de contenidos

- 1. [Visión general del proyecto](#)
- 2. [Stack tecnológico](#)
- 3. [Base de datos: entidades, tablas y relaciones](#)
- 4. [Catálogo y DataCatalogo](#)
- 5. [Roles, opciones y menú dinámico \(BD → JWT → Angular\)](#)
- 6. [Consulta dinámica y el patrón -1](#)
- 7. [Backend: arquitectura por capas](#)
- 8. [DTOs: Request vs Response](#)
- 9. [Mappers](#)
- 10. [Interfaz + Implementación \(Service / ServiceImpl \)](#)
- 11. [Anotaciones clave de Spring y JPA](#)
- 12. [Spring Security, JWT, Stateless y BCrypt](#)
- 13. [CORS y conceptos de red](#)
- 14. [Frontend: arquitectura y archivos raíz](#)
- 15. [Standalone components, Guards, Interceptors](#)
- 16. [Autenticación end-to-end](#)
- 17. [Patrones de diseño utilizados](#)
- 18. [Preguntas de entrevista técnica + respuestas](#)
- 19. [Glosario técnico](#)

1. Visión general del proyecto

MaintPro Industrial OS es un sistema de gestión de mantenimiento industrial para plantas que operan maquinaria crítica. Resuelve tres problemas reales:

Problema	Solución en MaintPro
No sé qué equipos tengo ni su estado	Módulo Activos (inventario + operativo / fuera de servicio)
Las fallas no se registran formalmente	Módulo Tickets (órdenes de trabajo vinculadas a un activo)
No hay visibilidad de KPIs	Dashboard (gráficos de disponibilidad y urgencia)

Flujo de negocio principal

Supervisor registra ticket sobre un activo

→ Activo pasa a FUERA DE SERVICIO (estado = 0)

→ Técnico actualiza estado del ticket (En Reparación → Cerrado)

→ Si no quedan tickets abiertos/en reparación → Activo vuelve a OPERATIVO (estado = 1)

Roles

Rol	Spring Security	Responsabilidad
Supervisor	ROLE_ADMIN	CRUD activos, registrar/eliminar tickets, dashboard, historial
Técnico	ROLE_TECH	Ver activos, ver y actualizar solo sus tickets asignados

2. Stack tecnológico

Backend

Tecnología	Versión	Función
Java	21	Lenguaje
Spring Boot	3.3.2	Framework REST
Spring Data JPA	—	ORM / repositorios
Spring Security	—	Autenticación y autorización
PostgreSQL	—	Base de datos relacional
JWT (jjwt)	0.11.5	Tokens stateless
Jakarta Validation	—	Validación en DTOs (@Valid, @NotBlank)
Lombok	—	Reducir boilerplate en entidades
springdoc-openapi	2.6.0	Swagger UI
Maven	—	Build

Frontend

Tecnología	Versión	Función
Angular	22	Framework SPA
TypeScript	6	Tipado estático
Angular Material	22	Componentes UI (no Bootstrap)
RxJS	7.8	Programación reactiva (Observables)
Chart.js + ng2-charts	4 / 10	Gráficos del dashboard
SweetAlert2	—	Alertas de éxito
MatSnackBar	—	Errores sobre modales
Vitest	—	Tests unitarios

Nota sobre Bootstrap: Este proyecto **no usa Bootstrap**. La UI se construye con **Angular Material** (componentes Material Design). Si escuchas "bootstrap" en Angular, suele referirse a `main.ts` que **arranca** (bootstrap) la

3. Base de datos: entidades, tablas y relaciones

Diagrama conceptual

```
usuario —< usuario_has_rol >— rol —< rol_has_opcion >— opcion

catalogo —< data_catalogo

data_catalogo —< activo (tipoActivo)
data_catalogo —< ticket_mantenimiento (prioridad, estadoTicket)

activo —< ticket_mantenimiento

usuario —< ticket_mantenimiento (tecnico, auditoria)
```

Tablas del sistema

Tabla	Entidad JPA	Propósito
usuario	Usuario	Credenciales y datos personales
rol	Rol	Perfiles (ROLE_ADMIN, ROLE_TECH)
opcion	Opcion	Ítems de menú (nombre, ruta Angular, tipo)
usuario_has_rol	—	N:N usuario ↔ rol
rol_has_opcion	—	N:N rol ↔ opción de menú
catalogo	Catalogo	Grupos de listas desplegables
data_catalogo	DataCatalogo	Valores concretos de cada catálogo
activo	Activo	Maquinaria / equipos
ticket_mantenimiento	TicketMantenimiento	Órdenes de mantenimiento

Relaciones importantes

Activo → Ticket (1:N)

Un activo puede tener muchos tickets históricos. FK: `ticket_mantenimiento.id_activo → activo.id_activo` . Impide borrar un activo si tiene tickets (integridad referencial).

DataCatalogo → Activo (N:1)

Cada activo tiene un `id_tipo_activo` (Compresor, Vehículo, etc.).

DataCatalogo → Ticket (N:1)

Cada ticket tiene `id_prioridad` y `id_estado_ticket` .

Usuario → Ticket (N:1)

Campo `id_usuario_tecnico` : técnico asignado.

Estados clave (constantes en `AppSettings.java`)

Concepto	Valor	Significado
Activo operativo	estado = 1	Disponible en planta
Activo fuera de servicio	estado = 0	Parado por falla/mantenimiento
Ticket abierto	id = 7	Recién registrado
Ticket en reparación	id = 8	Técnico trabajando
Ticket cerrado	id = 9	Historial

4. Catálogo y DataCatalogo

¿Por qué dos tablas?

Sin catálogo genérico tendrías:

```
tipo_activo, prioridad, estado_ticket → 3 tablas, 3 endpoints, 3 lógicas duplicadas
```

Con el patrón **Catálogo + DataCatalogo**:

catalogo (grupo)		data_catalogo (valores)
1 → Tipo de Activo	→	Maquinaria, Vehículos, Electrónicos...
2 → Prioridad	→	Baja, Media, Urgente
3 → Estado Ticket	→	Abierto, En Reparación, Cerrado

Ventajas

1. **Un solo endpoint genérico** por catálogo (`/util/listaTipoActivo` filtra `id_catalogo = 1`).
2. **Agregar un catálogo nuevo** = insertar fila en `catalogo` + valores en `data_catalogo` .
3. **Consistencia** en backend (`AppSettings.CATALOGO_TIPO_ACTIVO = 1`) y frontend (mismo ID).

En código

```
// Entidad Activo – relación ManyToOne
@ManyToOne
@JoinColumn(name = "id_tipo_activo")
private DataCatalogo tipoActivo;
```

El frontend recibe:

```
"tipoActivo": { "idDataCatalogo": 3, "descripcion": "Vehículos" }
```

5. Roles, opciones y menú dinámico (BD → JWT → Angular)

Modelo de seguridad en BD

usuario —< usuario_has_rol >— rol —< rol_has_opcion >— opcion

Tabla	Qué guarda
rol	ROLE_ADMIN, ROLE_TECH (formato exigido por Spring Security)
opcion	{ nombre, ruta, tipo } — la ruta coincide con app.routes.ts
rol_has_opcion	Qué módulos ve cada rol

Ejemplo en seed.sql

```
-- Opciones
(1, 'Gestión de Activos', 'activos', 3, 1),
(2, 'Tickets de Mantenimiento', 'tickets', 3, 1),
(4, 'Reportes y Dashboard', 'reportes', 4, 1);

-- Solo ROLE_ADMIN ve dashboard y usuarios (según rol_has_opcion en seed)
```

Flujo al login

1. POST /auth/login { login, password }
2. Spring Security valida BCrypt
3. UsuarioSeguridadServiceImpl carga roles + opciones desde BD
4. JwtProvider genera token con claims: roles[], opciones[]
5. Angular guarda token + opciones en sessionStorage
6. menu.ts filtra opciones por tipo (GESTIÓN vs REPORTES)
7. opcionGuard compara ruta actual con opciones[].ruta

¿Cómo limita la barra de Angular?

No está hardcodeda. El menú se construye desde el JWT:

```
// menu.ts
const opciones = this.tokenService.getOpciones();
this.opcMantenimiento = opciones.filter(x => x.tipo === 1 || x.tipo === 3);
```

Doble protección:

Capa	Mecanismo
UI	Botones/links solo si la opción está en el JWT
Router	opcionGuard bloquea navegación directa por URL
API	Spring Security exige JWT + roles en endpoints sensibles

Si un usuario sin rol no tiene opciones en BD → menú vacío + guards redirigen a `/`.

Alineación ruta BD ↔ Angular

BD opcion.ruta	app.routes.ts path	Componente
activos	activos	ActivoListaComponent
tickets	tickets	TicketRegistroComponent
reportes	reportes	DashboardComponent

El guard compara: `opciones.some(o => o.ruta === path)` .

6. Consulta dinámica y el patrón -1

Problema

Un mismo endpoint debe servir búsquedas con filtros **opcionales** sin crear 16 variantes del query.

Solución: sentinel -1 = "omitir filtro"

```
WHERE (?2 = -1 OR activo.id_activo = ?2)
AND (?3 = -1 OR prioridad.id = ?3)
```

En JPQL (`ActivoRepository`):

```
"(?3 = -1 or a.tipoActivo.idDataCatalogo = ?3) and " +
"(?4 = -1 or a.estado = ?4)"
```

En el frontend

```
// Si el usuario no elige filtro, enviamos -1
const params = new HttpParams()
  .set('vtipo', this.filtroTipo.toString()) // -1 = todos
  .set('vestado', this.filtroEstado.toString());
```

Tickets: filtro vpendientes

Valor	Comportamiento
1	Solo tickets abiertos (7) y en reparación (8) — pantalla operativa
0	Solo cerrados (9) — historial
-1	Todos los estados

Fechas

Parámetros `vfechaDesde` / `vfechaHasta` :string `yyyy-MM-dd` o `-1` .

En servicio se convierten a `LocalDateTime` ; si es `-1` , se usan fechas límite (1970 / 9999).

7. Backend: arquitectura por capas



Carpeta por carpeta

Paquete	Responsabilidad	Ejemplo
config/	Beans de configuración (OpenAPI/Swagger)	OpenApiConfig.java
controller/	REST API, HTTP, validación entrada	ActivoController
dto/	Contrato JSON Request/Response	ActivoCreateRequestDTO
entity/	Modelo persistente JPA	Activo, TicketMantenimiento
exception/	Manejo global de errores	GlobalExceptionHandler
mapper/	Traducción DTO ↔ Entity	ActivoMapper, TicketMapper
repository/	Acceso a datos	ActivoRepository extends JpaRepository
security/	JWT, filtros, SecurityConfig	JwtProvider, SecurityConfig
service/	Interfaces de negocio	ActivoService, TicketService
service/impl/	Implementación con @Service	ActivoServiceImpl, TicketServiceImpl
util/	Constantes globales	AppSettings (IDs catálogo, CORS)

¿Por qué separar capas?

- **Controller delgado:** solo HTTP + DTOs.
- **Service:** reglas de negocio reutilizables y testeables.
- **Repository:** SQL/JSQL aislado.
- **Entity:** no sale del backend (evita acoplar API a tablas).

8. DTOs: Request vs Response

DTO = Data Transfer Object. Objeto que define el **contrato de la API**, independiente de la base de datos.

Request DTO (entrada)

Lo que el cliente **envía**. Validado con `@Valid` :

DTO	Cuándo	Campos clave
ActivoCreateRequestDTO	POST	nombre, numeroSerie, idTipoActivo, idUsuarioRegistro
ActivoUpdateRequestDTO	PUT	idActivo, nombre, numeroSerie, idTipoActivo, estado
TicketCreateRequestDTO	POST	descripcion, idActivo, idPrioridad, idEstadoTicket...
TicketUpdateRequestDTO	PUT	idTicket + campos opcionales según rol
LoginRequestDTO	POST login	login, password

¿Por qué Create y Update separados?

- Create **no** lleva `idActivo` (lo genera la BD).
- Update **exige** `idActivo` y campos como `estado` .
- Validaciones distintas (`@NotNull` en campos diferentes).

Response DTO (salida)

Lo que el cliente **recibe**. Evita exponer entidades JPA:

DTO	Cuándo
ActivoResponseDTO	GET listas — incluye tipoActivo anidado, auditoría
TicketResponseDTO	GET tickets — activo resumido, prioridad, técnico
JwtResponseDTO	Login — token, roles, opciones
MensajeDTO	POST/PUT/DELETE exitosos — { "mensaje": "..."} }
DataCatalogoDTO	Catálogos util
UsuarioResumenDTO	Lista técnicos — sin password

Ejemplo en controller

```
@PostMapping("/registraActivo")
public ResponseEntity<MensajeDTO> registra(@Valid @RequestBody ActivoCreateRequestDTO dto) {
    Activo entity = activoMapper.toEntity(dto);
    Activo saved = activoService.insertaActualizaActivo(entity);
    return ResponseEntity.ok(new MensajeDTO("Activo '" + saved.getNombre() + "'
registrado."));
}
```

El controller nunca menciona `Activo` en la firma pública.

9. Mappers

Mapper = traductor entre DTO y Entity.

```
@Component
public class ActivoMapper {
    public Activo toEntity(ActivoCreateRequestDTO dto) {
        Activo a = new Activo();
        a.setNombre(dto.nombre().trim());
        a.setTipoActivo(referenciaCatalogo(dto.idTipoActivo()));
        return a;
    }

    public ActivoResponseDTO toResponse(Activo activo) {
        return new ActivoResponseDTO(
            activo.getIdActivo(),
            activo.getNombre(),
            toCatalogoDto(activo.getTipoActivo()),
            ...
        );
    }
}
```

¿Por qué no mapear en el controller?

- **Single Responsibility:** controller no conoce estructura interna de Entity.
- **Reutilización:** mismo mapper en create, update, list.
- **Referencias JPA:** mapper crea objetos ligeros { id: 5 } sin cargar toda la fila.

10. Interfaz + Implementación (Service / ServiceImpl)

```
// Interfaz - contrato
public interface TicketService {
    TicketMantenimiento registraTicket(TicketMantenimiento obj);
    List<TicketMantenimiento> consultaDinamica(...);
}

// Implementación - lógica real
@Service
public class TicketServiceImpl implements TicketService {
    @Override
    @Transactional
    public TicketMantenimiento registraTicket(TicketMantenimiento obj) { ... }
}
```

¿Por qué @Service en la impl y no en la interfaz?

Spring escanea **clases concretas** con anotaciones. La interfaz es solo contrato; Spring crea un **bean** de TicketServiceImpl y lo inyecta donde se pide TicketService .

Ventajas del patrón

1. **Desacoplamiento:** controller depende de `TicketService` , no de `TicketServiceImpl` .
2. **Testing:** puedes mockear la interfaz en tests.
3. **Claridad:** interfaz documenta qué operaciones expone el módulo.

11. Anotaciones clave de Spring y JPA

Anotación	Dónde	Para qué
<code>@RestController</code>	Controller	Combina <code>@Controller</code> + <code>@ResponseBody</code> (JSON automático)
<code>@RequestMapping</code>	Controller	Prefijo URL (<code>/url/activo</code>)
<code>@GetMapping</code> / <code>@PostMapping</code> / <code>@PutMapping</code> / <code>@DeleteMapping</code>	Métodos	Verbo HTTP
<code>@PathVariable</code>	Parámetro	{id} en URL
<code>@RequestParam</code>	Parámetro	Query string (<code>?vnombre=</code>)
<code>@RequestBody</code>	Parámetro	JSON del body
<code>@Valid</code>	Parámetro DTO	Dispara Jakarta Validation
<code>@Autowired</code>	Campo/constructor	Inyección de dependencias (DI)
<code>@Service</code>	ServiceImpl	Bean de lógica de negocio
<code>@Repository</code>	(<i>implícito en JpaRepository</i>)	Bean de acceso a datos
<code>@Transactional</code>	Método service	Todo-o-nada en BD
<code>@Override</code>	Método	Indica que implementas método de interfaz/clase padre
<code>@Entity</code>	Clase	Mapeo JPA a tabla
<code>@Table(name="activo")</code>	Entidad	Nombre de tabla
<code>@Id</code> / <code>@GeneratedValue</code>	Campo	Primary key autoincremental
<code>@ManyToOne</code> / <code>@JoinColumn</code>	Campo	FK relación N:1
<code>@PrePersist</code> / <code>@PreUpdate</code>	Método entidad	Auditoría automática de fechas
<code>@Configuration</code>	SecurityConfig	Clase de configuración Spring
<code>@Bean</code>	Método config	Registra bean manual (filtro JWT, encoder)

@Override

Garantiza que estás sobrescribiendo un método existente. Si la interfaz cambia, el compilador avisa.

@Autowired

Spring inyecta la dependencia automáticamente (Inversión de Control). No haces `new ActivoServiceImpl()` .

12. Spring Security, JWT, Stateless y BCrypt

Spring Security — cadena de filtros

Request HTTP

- `JwtTokenFilter` (lee header `Authorization: Bearer ...`)
- `SecurityFilterChain` (evalúa reglas: `permitAll`, `hasAuthority...`)
- `Controller`

`SecurityConfig.java` define:

- Rutas públicas: `/auth/**` , Swagger
- Rutas por rol: `POST /activo/**` → `ROLE_ADMIN`
- CORS para `localhost:4200`
- CSRF desactivado (API REST)
- `SessionCreationPolicy.STATELESS`

¿Qué es Stateless?

Sin estado en servidor. Tradicionalmente la sesión se guardaba en memoria del servidor (session ID en cookie).

Con JWT:

- El servidor **no guarda** sesión.
- Toda la info necesaria va en el **token firmado**.
- Cualquier instancia del backend puede validar el token.

Ventaja: escalabilidad horizontal (varios servidores sin compartir sesión).

JWT (JSON Web Token)

Estructura: `header.payload.signature`

Payload incluye (claims):

```
{
  "idUsuario": 1,
  "nombreCompleto": "Supervisor MaintPro",
  "roles": ["ROLE_ADMIN"],
  "opciones": [{ "ruta": "activos", "nombre": "Gestión de Activos" }],
  "sub": "admin",
  "exp": 1734567890
}
```

Flujo:

1. Login exitoso → `JwtProvider.generateToken()`
2. Cliente guarda token
3. Cada request: `Authorization: Bearer eyJhbG...`
4. `JwtTokenFilter` valida firma + expiración
5. Crea `Authentication` en `SecurityContextHolder`

Secret en `application.properties` → `jwt.secret` (no hardcodeado en código).

BCrypt

Algoritmo de hash **unidireccional** para contraseñas:

```
"admin2026" → $2a$10$IUWTjcD7NQnQBcQn0oSZYu...
```

- Incluye **salt** aleatorio (misma password → hashes distintos).
- Spring `BCryptPasswordEncoder` en login compara hash almacenado vs password ingresada.
- **Nunca** se guarda la contraseña en texto plano.

13. CORS y conceptos de red

CORS (Cross-Origin Resource Sharing)

Problema: Frontend en `http://localhost:4200` llama API en `http://localhost:8080` → **orígenes distintos**.
El navegador bloquea por seguridad unless el servidor autoriza.

Solución en MaintPro:

```
corsConfiguration.setAllowedOrigins(List.of("http://localhost:4200"));  
corsConfiguration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
```

REST / API Stateless

- **REST:** recursos identificados por URL, verbos HTTP semánticos.
- **Stateless:** cada request lleva credenciales (JWT), servidor no recuerda sesión anterior.

HTTP Status usados

Código	Cuándo
200	OK
400	Validación fallida (@valid)
401	Sin token o token inválido
403	Token válido pero rol insuficiente
409	Conflicto (SN duplicado, FK al eliminar)
500	Error interno (mensaje genérico al cliente)

14. Frontend: arquitectura y archivos raíz

Estructura de carpetas

```
src/app/  
├─ auth/login/      → Pantalla login  
├─ components/      → Módulos funcionales (activos, tickets, dashboard)  
├─ guards/          → opcionGuard, adminGuard
```

interceptors/	→ JWT Bearer automático
menu/	→ Barra navegación dinámica
models/	→ Interfaces TypeScript (Activo, Ticket...)
security/	→ TokenService, AuthService
services/	→ HTTP hacia backend (mapeo DTO)
utils/	→ Helpers (activo-feedback snackbar)
app.routes.ts	→ Rutas + guards
app.config.ts	→ Providers globales
app.settings.ts	→ URL del API
app.ts	→ Componente raíz
app.html	→ Shell layout
app.css	→ Estilos shell

Archivos raíz explicados

Archivo	Función
app.html	Shell de la app: <app-menu> + <router-outlet> + footer. No cambia entre pantallas.
app.ts	Componente raíz que carga el template.
app.css	Estilos del layout principal.
app.routes.ts	Define rutas y qué componente/guard usa cada URL.
app.config.ts	Registra providers: router, HTTP, interceptor, animaciones, charts.
app.settings.ts	Constante API_ENDPOINT = 'http://localhost:8080/url'.
main.ts	Bootstrap de Angular (arranca la app en el DOM).
styles.css	Tema global industrial + overrides Material + z-index SweetAlert.

¿Por qué solo <app-menu> en app.html?

Patrón **Layout Shell**:

```
<app-menu></app-menu>           <!-- Siempre visible si hay sesión -->
<router-outlet></router-outlet> <!-- Aquí se monta Activos, Tickets, etc. -->
```

El menú es persistente; el contenido cambia según la ruta.

15. Standalone components, Guards, Interceptors

Standalone Components (Angular 22)

Antes necesitabas `NgModule`. Ahora cada componente declara sus imports:

```
@Component({
  standalone: true,
  imports: [CommonModule, MatButtonModule, ...],
```

```
    templateUrl: './activo-lista.html'
  })
```

Ventaja: menos boilerplate, lazy loading más simple.

Guards — protección de rutas

Guard	Función
opcionGuard	¿Hay token? ¿La ruta está en opciones[] del JWT?
adminGuard	¿Tiene ROLE_ADMIN? (historial, rutas exclusivas)

```
export const opcionGuard: CanActivateFn = (route) => {
  const path = route.routeConfig?.path; // ej: 'activos'
  return tokenService.getOpciones().some(o => o.ruta === path);
};
```

Interceptor — JWT automático

```
intercept(req, next) {
  if (token) {
    req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` }});
  }
  return next.handle(req);
}
```

Todos los `HttpClient.get/post` llevan el token sin repetir código en cada service.

Components

Un **componente** = unidad UI (HTML + TS + CSS):

- `ActivoListaComponent` → tabla + filtros
- `ActivoAddComponent` → diálogo modal crear
- `TicketRegistroComponent` → tickets pendientes

Comunicación con backend vía **Services** inyectados.

16. Autenticación end-to-end

1. Usuario ingresa login/password en `LoginComponent`
2. `AuthService` → POST `/auth/login` (`LoginRequestDTO`)
3. Backend: `AuthenticationManager` + `BCrypt`
4. Response: `JwtResponseDTO { token, roles, opciones }`
5. `TokenService` guarda en `sessionStorage`
6. Router navega a `/`
7. `MenuComponent` lee opciones → construye navbar
8. Usuario click "Activos" → router `/activos`
9. `opcionGuard` valida `'activos' ∈ opciones`

```
10. ActivolistaComponent carga → ActivoService.consultaDinamica()  
11. Interceptor adjunta Bearer token  
12. Backend valida JWT + devuelve ActivoResponseDTO[]
```

17. Patrones de diseño utilizados

Patrón	Dónde	Beneficio
Layered Architecture	Backend completo	Separación de responsabilidades
DTO	controller/dto	Contrato API estable
Mapper	mapper/	Traducción entre capas
Repository	Spring Data JPA	Abstracción de persistencia
Dependency Injection	@Autowired	Bajo acoplamiento
Front Controller	Spring @RestController	Punto único de entrada HTTP
Filter Chain	Spring Security + JWT	Cross-cutting security
Guard	Angular	Autorización en router
Interceptor	Angular HTTP	Cross-cutting (token)
Master-Detail	catalogo/data_catalogo	Reutilización de catálogos
Sentinel Value	consulta -1	Filtros opcionales sin null complejo

18. Preguntas de entrevista técnica + respuestas

Proyecto y negocio

P: ¿Qué problema resuelve MaintPro?

R: Centraliza inventario de activos industriales y tickets de mantenimiento, con trazabilidad entre falla y disponibilidad del equipo, más KPIs para supervisores.

P: ¿Por qué un ticket pone el activo fuera de servicio?

R: Regla de negocio: si hay una orden abierta o en reparación, el equipo no está disponible operativamente. Al cerrar todos los tickets activos, vuelve a operativo.

P: ¿Por qué el técnico no registra tickets?

R: Separación de roles: el supervisor planifica y asigna; el técnico ejecuta y actualiza estado.

Base de datos

P: ¿Por qué Catálogo y DataCatalogo y no tablas separadas?

R: Evita duplicar estructura y endpoints. Un patrón genérico sirve tipos de activo, prioridades y estados.

P: ¿Qué pasa si intento borrar un activo con tickets?

R: FK de PostgreSQL lo impide. API responde 409. Decisión de diseño: preservar historial.

Backend

P: ¿Por qué DTOs si ya tienes entidades?

R: La entidad refleja la BD (relaciones JPA, lazy loading). El DTO define qué entra y sale por HTTP, con validación y sin filtrar datos internos.

P: ¿Diferencia entre @Controller y @RestController ?

R: @RestController serializa la respuesta a JSON automáticamente (@ResponseBody en cada método).

P: ¿Qué hace @Transactional ?

R: Si ocurre error mid-operation, hace rollback de todos los cambios BD en ese método.

P: ¿Por qué @Service en la impl?

R: Spring registra beans sobre clases concretas. La interfaz es el contrato para inyectar.

Seguridad

P: ¿Qué es stateless y por qué lo usas?

R: El servidor no guarda sesión HTTP. El JWT lleva identidad y permisos. Escala mejor y es típico en APIs REST.

P: ¿Cómo proteges endpoints sin tocar cada controller?

R: SecurityFilterChain en SecurityConfig + @PreAuthorize alternativo; nosotros usamos requestMatchers por URL y método HTTP.

P: ¿BCrypt es encriptación?

R: No, es **hash** one-way. No se puede recuperar la password original. Login compara hash almacenado vs hash de input.

P: ¿Dónde validas roles además de SecurityConfig?

R: En servicios (TicketServiceImpl valida ticket asignado al técnico) — defensa en profundidad.

Frontend

P: ¿Cómo el menú se sincroniza con la BD?

R: Login devuelve opciones[] en JWT desde rol_has_opcion . Menu y guards leen eso — no hay rutas hardcodeadas por rol en Angular.

P: ¿Qué es un Guard?

R: Función que Angular ejecuta **antes** de activar una ruta. Retorna true/false para permitir o redirigir.

P: ¿Interceptor vs Guard?

R: Guard protege **rutas/navegación**. Interceptor modifica **peticiones HTTP** (añade token).

P: ¿Usas Bootstrap?

R: No. Angular Material. "Bootstrap" en main.ts significa **iniciar** la aplicación.

Consulta dinámica

P: ¿Por qué -1 y no null?

R: Parámetros primitivos int en Java no aceptan null. -1 es convención del proyecto = filtro omitido. Consistente en frontend y backend.

Mejoras futuras

P: ¿Qué le falta para producción?

R: Tests automatizados, perfiles dev/prod, interceptor 401 global, logging estructurado, CI/CD, soft delete en activos.

19. Glosario técnico

Término	Definición
API REST	Interfaz HTTP con recursos y verbos (GET/POST/PUT/DELETE)
BCrypt	Algoritmo hash para contraseñas con salt
Bearer Token	Esquema HTTP: Authorization: Bearer <jwt>
CORS	Permiso del navegador para llamadas cross-origin
CRUD	Create, Read, Update, Delete
DTO	Objeto de transferencia de datos (contrato API)
Entity	Clase JPA mapeada a tabla SQL
FK	Foreign Key — integridad referencial
Guard	Protección de rutas en Angular
Interceptor	Middleware HTTP en Angular
JWT	Token firmado JSON stateless
JPQL	Query orientado a objetos JPA
Lazy Loading	Carga diferida de relaciones JPA
Mapper	Convierte DTO ↔ Entity
ORM	Object-Relational Mapping (JPA/Hibernate)
RBAC	Role-Based Access Control
Repository	Capa acceso a datos Spring Data
Sentinel Value	Valor especial (-1) con significado convencional
SPA	Single Page Application (Angular)
Standalone	Componentes Angular sin NgModule
Stateless	Servidor sin sesión; estado en token
Swagger	Documentación interactiva OpenAPI

Pitch de 30 segundos (memorizar)

"Desarrollé MaintPro, un sistema fullstack de mantenimiento industrial con Spring Boot y Angular. Implementé RBAC supervisor/técnico con menú dinámico desde PostgreSQL, trazabilidad activo-ticket, API con capa DTO y validación Jakarta, JWT stateless con BCrypt, y consultas dinámicas reutilizables. El frontend usa guards alineados con las opciones del token, interceptors JWT y Angular Material."